
LLaVA
VGGT
Spatial Stack
paper review

2026. 09. 21

이준형

PART I. Visual Instruction Tuning (LLaVA)

- Introduction
- GPT-assisted Data Generation
- Architecture & Training
- Experiments
- Conclusion

PART II. VGGT: Visual Geometry Grounded Transformer

- Motivation
- Architecture

PART III. SpatialStack : Layered Geometry-Language Fusion for 3D VLM Spatial Reasoning

- Introduction
- Why Multi-level Geometry Features?
- Architecture: Geometry-Language Fusion
- Experiments
- Conclusion

PART I. Visual Instruction Tuning (LLaVA)

paper review

목차

- Introduction
- **GPT-assisted Data Generation**
- **Architecture & Training**
- Experiments
- Conclusion

❖ Visual Instruction Tuning

2023년 NeurIPS에서 발표

현재 멀티모달 LLM(VLM) 분야에서 가장 많이 인용되는 대표 논문 중 하나

*Google Scholar 기준 16,984회 인용 (2026년 8월 기준)

GPT-4를 활용해 멀티모달 instruction-following 데이터를 생성하고, 이를 Vision-Language 모델 학습에 **최초로 적용**한 논문

Visual Instruction Tuning

Haotian Liu^{1*}, Chunyuan Li^{2*}, Qingyang Wu³, Yong Jae Lee¹

¹University of Wisconsin–Madison ²Microsoft Research ³Columbia University
<https://llava-vl.github.io>

Abstract

Instruction tuning large language models (LLMs) using machine-generated instruction-following data has been shown to improve zero-shot capabilities on new tasks, but the idea is less explored in the multimodal field. We present the first attempt to use language-only GPT-4 to generate multimodal language-image instruction-following data. By instruction tuning on such generated data, we introduce LLaVA: **L**arge **L**anguage and **V**ision **A**ssistant, an end-to-end trained large multimodal model that connects a vision encoder and an LLM for general-purpose visual and language understanding. To facilitate future research on visual instruction following, we construct two evaluation benchmarks with diverse and challenging application-oriented tasks. Our experiments show that LLaVA demonstrates impressive multimodal chat abilities, sometimes exhibiting the behaviors of multimodal GPT-4 on unseen images/instructions, and yields a 85.1% relative score compared with GPT-4 on a synthetic multimodal instruction-following dataset. When fine-tuned on Science QA, the synergy of LLaVA and GPT-4 achieves a new state-of-the-art accuracy of 92.53%. We make GPT-4 generated visual instruction tuning data, our model, and code publicly available.

Introduction

- Motivation

❖ 만약 인공지능(AI)이 이미지를 보고 자유롭게 대화할 수 있다면?

- 인간의 눈을 대체할 수 있는 시각 기능 (이미지 안에 무엇이 있는지)
- 시각 정보를 언어로 풀어내는 언어 능력 (자연스러운 대화 형태)
- 사용자의 다양한 지시를 유연하게 따르는 지시 이행 능력



이 “?”를 채우는 것이 *Visual Instruction Tuning*의 목표

Introduction

- Background

❖ 기존 비전 모델의 한계 vs LLM이 보여준 가능성

기존 비전 모델

- 분류 · 탐지 · 세그멘테이션 등 태스크별로 개별 학습
- 태스크 지시가 모델 설계에 암묵적으로 고정됨
- 사용자 지시에 유연하게 반응하지 못함



LLM (ChatGPT / GPT-4)

- 언어를 '범용 인터페이스'로 사용
- 다양한 태스크 지시를 언어로 명시적으로 표현 가능
- 하나의 모델이 여러 지시를 유연하게 수행

→ LLM의 instruction-following 능력을 이미지-언어 멀티모달 영역으로 확장하자



Introduction

- Contributions of this paper

❖ 이 논문의 4가지 핵심 기여

①	Multimodal instruction data	GPT-4를 활용해 이미지-텍스트 쌍을 instruction 형식으로 변환하는 파이프라인 제안
②	Large Language and Vision Assistant	CLIP 비전 인코더 + Vicuna LLM을 연결한 end-to-end 모델
③	LLaVA-Bench	다양한 태스크로 구성된 평가 벤치마크 2종 제안
④	전면 오픈소스	데이터 · 코드 · 모델 체크포인트 · 데모 전체 공개

GPT-assisted Data Generation

- Motivation for data pipeline

❖ 가장 큰 문제: 비전-언어 instruction 데이터의 부재

- 이미지-텍스트 쌍(캡션) 데이터는 많지만, '지시-응답' 형태의 데이터는 거의 없음
- GPT-4/ChatGPT를 '데이터 생성기'로 활용하는 아이디어

이미지를 텍스트만 받는 GPT-4에게 어떻게 전달할까?

기호적 표현 1

Captions

장면을 다양한 관점에서 서술한 문장들

기호적 표현 2

Bounding Boxes

객체의 종류와 이미지 내 위치 좌표

Context type 1: Captions

A group of people standing outside of a black vehicle with various luggage.

Luggage surrounds a vehicle in an underground parking area

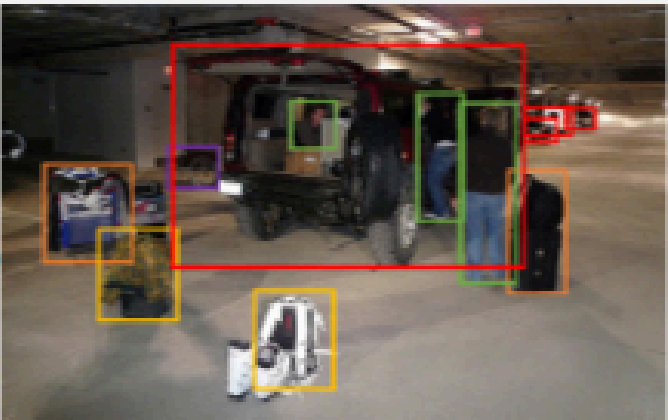
People try to fit all of their luggage in an SUV.

The sport utility vehicle is parked in the public garage, being packed for a trip

Some people with luggage near a van that is transporting it.

Context type 2: Boxes

person: [0.681, 0.242, 0.774, 0.694], backpack: [0.384, 0.696, 0.485, 0.914], suitcase: ...<omitted>



→ 이 두 가지를 텍스트로 GPT-4에 입력해 instruction 데이터를 자동 생성

GPT-assisted Data Generation

- Why text-only input

❖ 왜 이미지 대신 텍스트(Caption+Bounding Box)만으로 GPT-4에 전달했을까?

구조적 제약

- GPT-4/ChatGPT는 텍스트 전용 모델
→ 이미지를 직접 처리할 수 없음
- 이미지를 캡션("A black SUV surrounded by luggage")과 bounding box(좌표)로 텍스트화해서 우회 입력

비용 효율성

- 이미지 기반 데이터를 사람이 직접 라벨링하면 비용·시간이 크게 소요
- 기존 텍스트 캡션 데이터셋을 재활용하면 대규모 데이터를 빠르고 저렴하게 증강 가능

→ GPT-4의 강력한 언어 추론 능력만으로도, caption·bounding box 정보만 가지고 고품질 질문-응답 데이터 생성 가능

GPT-assisted Data Generation

- Three types of generated data

❖ 생성된 Instruction 데이터의 3가지 유형

대화형 (Conversation)

58K

이미지에 대한 자연스러운 Q&A.
객체 종류·개수·위치·상호작용 등에 대한 질의응답

상세 설명 (Detailed description)

23K

이미지를 풍부하고 포괄적으로 서술하는 응답

복합 추론 (Complex reasoning)

77K

단계적 논리 전개가 필요한 심층 질의응답

Response type 1: conversation

Question: What type of vehicle is featured in the image?
Answer: The image features a black sport utility vehicle (SUV) ...<omitted>

Response type 2: detailed description

The image is an underground parking area with a black sport utility vehicle (SUV) parked. There are three people in the scene, with one person standing closer to the left side of the vehicle, another person in the middle, and the third person on the right side. They are all working together to pack their luggage into the SUV for a trip. ...<omitted>

Response type 3: complex reasoning

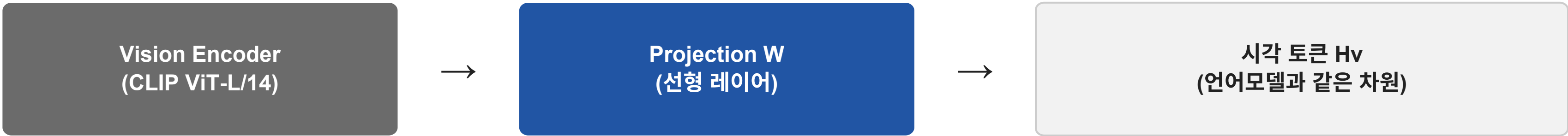
Question: What challenges do these people face?
Answer: In the image, a group of people is standing outside a black SUV in a parking area, surrounded by various pieces of luggage, including suitcases and backpacks. They are facing the challenge of fitting all their luggage into the black SUV. There are multiple suitcases and backpacks to be packed, which suggests that the group has a significant amount of belongings ...<omitted>

총 158K개의 *language-image instruction* 샘플 (COCO 이미지 기반)

Architecture

- Step 1 — Vision encoding & projection

❖ 이미지를 언어모델이 이해할 수 있는 토큰으로 변환하는 과정



수식으로 표현하면

$$H_v = W \cdot Z_v, \quad Z_v = \text{CLIP}(X_v)$$

- Z_v : 이미지 X_v 를 CLIP으로 인코딩한 시각특징
- W : 학습 가능한 projection matrix (선형 변환)
- H_v : 언어모델의 word embedding과 같은 차원의 시각 토큰

설계 포인트

의도적으로 단순한 선형 레이어만 사용.
→ 빠른 데이터 중심 실험 반복 선택

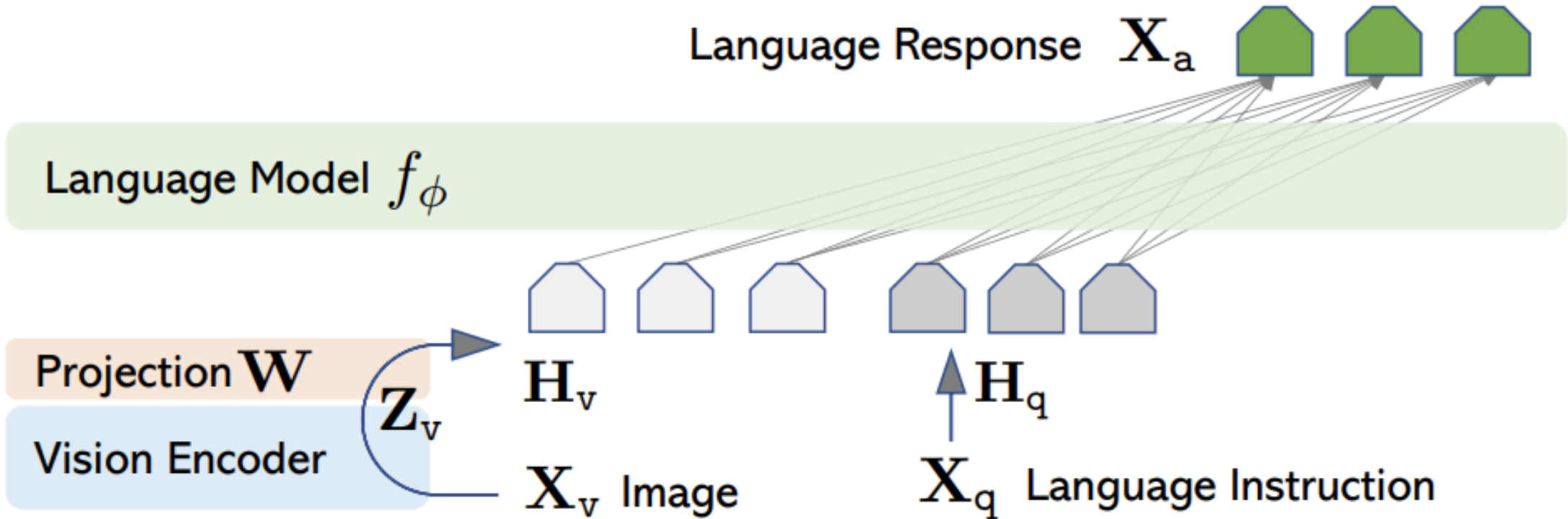
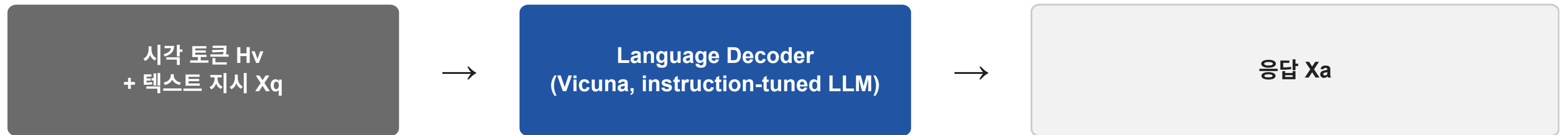


Figure 1: LLaVA network architecture.

Architecture

- Step 2 — Language decoding

❖ 시각 토큰과 텍스트를 하나의 시퀀스로 결합해 응답을 생성하는 과정



왜 Vicuna인가?

- instruction-tuned LLM
 - 다양한 지시를 유연하게 따르는 능력이 사전에 이미 확보됨
- GPT 계열과 유사한 decoder-only 구조
 - 별도의 멀티모달 전용 구조 없이 기존 LLM을 거의 그대로 재사용

어떻게 결합하는가?

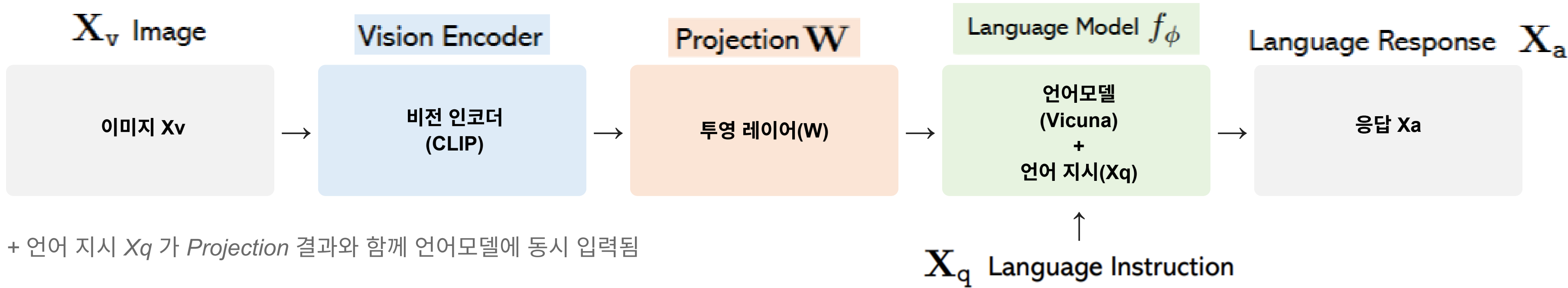
- Hv(시각 토큰)와 텍스트 토큰을 동일한 시퀀스 안에 함께 배치 (언어모델 입장에서 이미지는 '특수한 토큰들'일 뿐)
- 예: “이미지 설명: ” + Hv
→ 이어서 자연어 응답 Xa를 auto-regressive하게 생성

언어 토큰과 비주얼 토큰을 같은 시퀀스로 처리 → 텍스트 생성뿐 아니라 질문-응답, 복합 추론까지 하나의 decoder로 통합 수행



Architecture

❖ LLaVA 전체 네트워크 구조



Training

- Instruction sequence construction

❖ Training Data구축

```
Xsystem-message <STOP>
Human : Xinstruct1 <STOP> Assistant: Xa1 <STOP>
Human : Xinstruct2 <STOP> Assistant: Xa2 <STOP> ...
```

입력 시퀀스 형식 (multi-turn 대화)

- 각 Image X_v 에 대해, Multi-Turn Conversation Data($X_q^1, X_a^1, \dots, X_q^T, X_a^T$) 확보
- 각 Conversation의 답변을 Assistant의 답변으로 간주
- t번째 Instruction은 아래와 같이 지정

$$X_{\text{instruct}}^t = \begin{cases} \text{Randomly choose } [X_q^1, X_v] \text{ or } [X_v, X_q^1], & \text{the first turn } t = 1 \\ X_q^t, & \text{the remaining turns } t > 1 \end{cases}$$

<The Instruction at t-th Turn>

일관된 형태로 Multimodal Instruction-Following Sequence 형성가능



Training

- Loss function breakdown

❖ Probability 계산

$X_{\text{system-message}} <\text{STOP}>$
Human : $X_{\text{instruct}}^1 <\text{STOP}>$ Assistant: $X_a^1 <\text{STOP}>$
Human : $X_{\text{instruct}}^2 <\text{STOP}>$ Assistant: $X_a^2 <\text{STOP}> \dots$

입력 시퀀스 형식 (multi-turn 대화)

- Sequence의 길이가 L일때, 정답X에 대한 확률은 아래와 같이계산
 - ※ $X_{\text{instruct}, <i}$:현재 예측 토큰인 X_i 이전 모든 Turns에 대한 Instruction Tokens
 - ※ $X_{a, <i}$:현재 예측 토큰인 X_i 이전 모든 Turns에 대한 Answer Tokens

$$p(\mathbf{X}_a | \mathbf{X}_v, \mathbf{X}_{\text{instruct}}) = \prod_{i=1}^L p_{\theta}(\mathbf{x}_i | \mathbf{X}_v, \mathbf{X}_{\text{instruct}, <i}, \mathbf{X}_{a, <i})$$

→ 학습 파라미터

< The Probability of Target Answers X_a >

- Auto-Regressive Model에서 초록색 부분의 Sequence/Tokens만 Loss 계산에이용

Training

❖ Stage 1: Feature Alignment 사전학습

- CC3M을 필터링한 595K개의 이미지-텍스트 쌍 사용
- 비전 인코더(CLIP) + LLM(Vicuna) 가중치는 모두 고정(freeze)
- 오직 Projection matrix (W)만 학습

의미: 고정된 LLM을 위한 '호환 가능한 시각 토크나이저'를 학습하는 과정

❖ Stage 2: End-to-End 파인튜닝

- 비전 인코더는 계속 고정, Projection + LLM 전체를 함께 업데이트

$\theta = \{ W, \phi_{\text{LLM}} \}$ (projection + LLM 전체)

ϕ_{LLM} : Vicuna LLM 파라미터

멀티모달 챗봇

158K instruction 데이터 전체로 파인튜닝. 대화형은 multi-turn, 나머지 두 유형은 single-turn

ScienceQA

이미지+텍스트 맥락이 주어진 객관식 과학 문제. 추론 과정과 정답을 함께 생성하도록 학습

전체 목적함수: 자기회귀적(auto-regressive) 다음 토큰 예측, 정답 토큰에 대해서만 loss 계산

Experiments

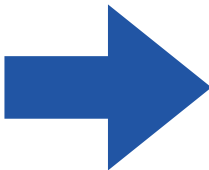
- Qualitative Example

❖ 실제 답변으로 보는 LLaVA의 이해력

- 정량 점수만으로는 체감되지 않는 차이를, 실제 프롬프트에 대한 4개 모델의 답변으로 비교
- GPT-4 논문에서 가져온 동일한 프롬프트로 LLaVA의 추론·설명 능력을 테스트



Q. What is unusual about this image?



Model	답변
LLaVA	미니밴 뒤에서 남자가 옷을 다림질하고 있으며, 일반적이지 않은 장소·불안정한 자세라는 점까지 짚어냄
GPT-4	움직이는 택시 지붕에 부착된 다림질판에서 옷을 다림질하고 있다는 점을 지적
BLIP-2	"노란 택시 뒤에 앉아있는 남자" → 질문 의도(무엇이 특이한가)엔 답하지 못하고 단순 묘사에 그침
OpenFlamingo	"차 후드 위에서 옷을 말리고 있는 남자" → 다림질을 건조로 오인, 사실관계 자체가 틀림

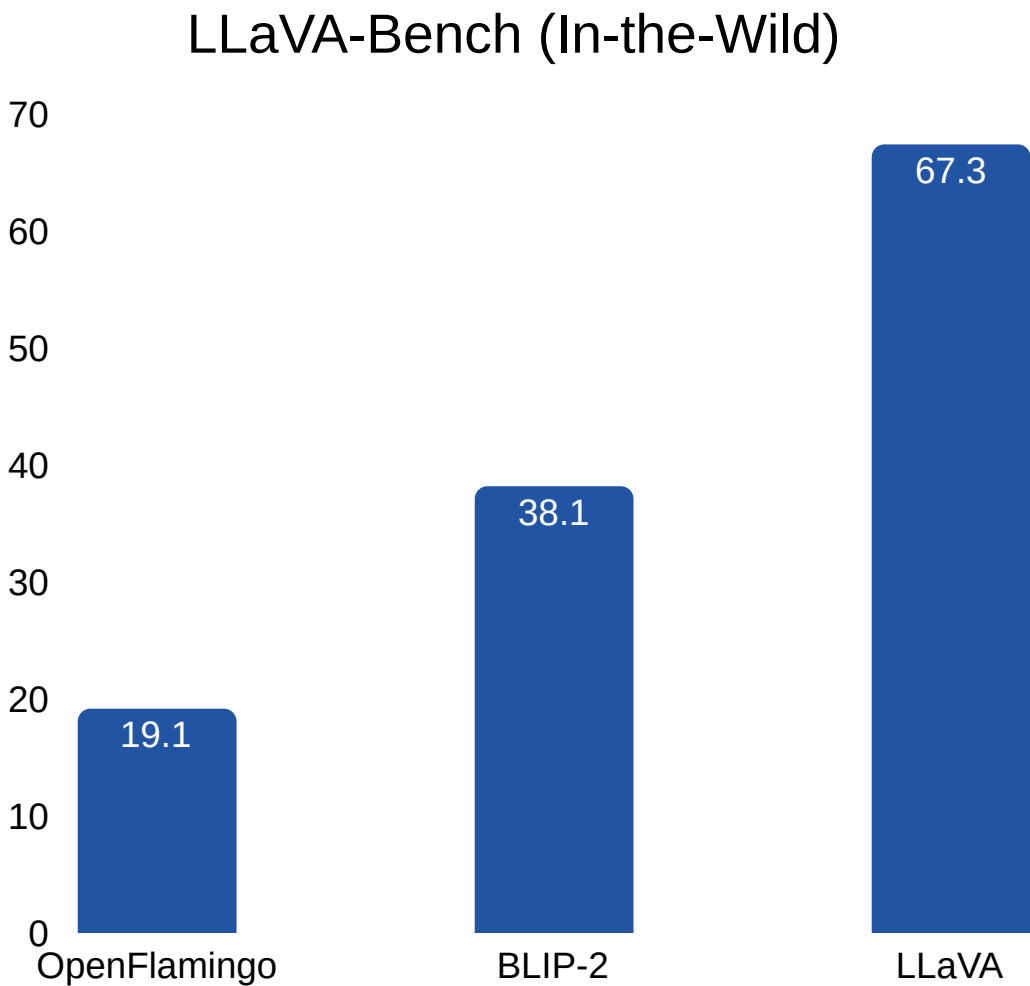
80K개 정도의 적은 이미지로 학습했음에도 GPT-4와 유사한 심층 답변 — 단순 묘사에 그치는 BLIP-2·OpenFlamingo와 달리, 질문 의도에 맞춰 답함

Experiments

- LLaVA-Bench evaluation

- ❖ 정량 평가: GPT-4를 심판으로 활용한 상대 점수
- 텍스트만 보는 GPT-4의 답변을 이론적 상한선으로 삼아 상대 점수 산출

	Conversation	Detail description	Complex reasoning	All
Full data	83.1	75.3	96.5	85.1
Detail + Complex	81.5 (-1.6)	73.3 (-2.0)	90.8 (-5.7)	81.9 (-3.2)
Conv + 5% Detail + 10% Complex	81.0 (-2.1)	68.4 (-7.1)	91.5 (-5.0)	80.5 (-4.4)
Conversation	76.5 (-6.6)	59.8 (-16.2)	84.9 (-12.4)	73.8 (-11.3)
No Instruction Tuning	22.0 (-61.1)	24.0 (-51.3)	18.5 (-78.0)	21.5 (-63.6)



BLIP-2 대비 1.8배(38.1→67.3), OpenFlamingo 대비 3.5배(19.1→67.3) 높은 성능을 적은 데이터로 달성. 다만 이미지를 '패치 뭉치'로 인식해 세밀한 의미를 놓치는 한계도 관찰됨.

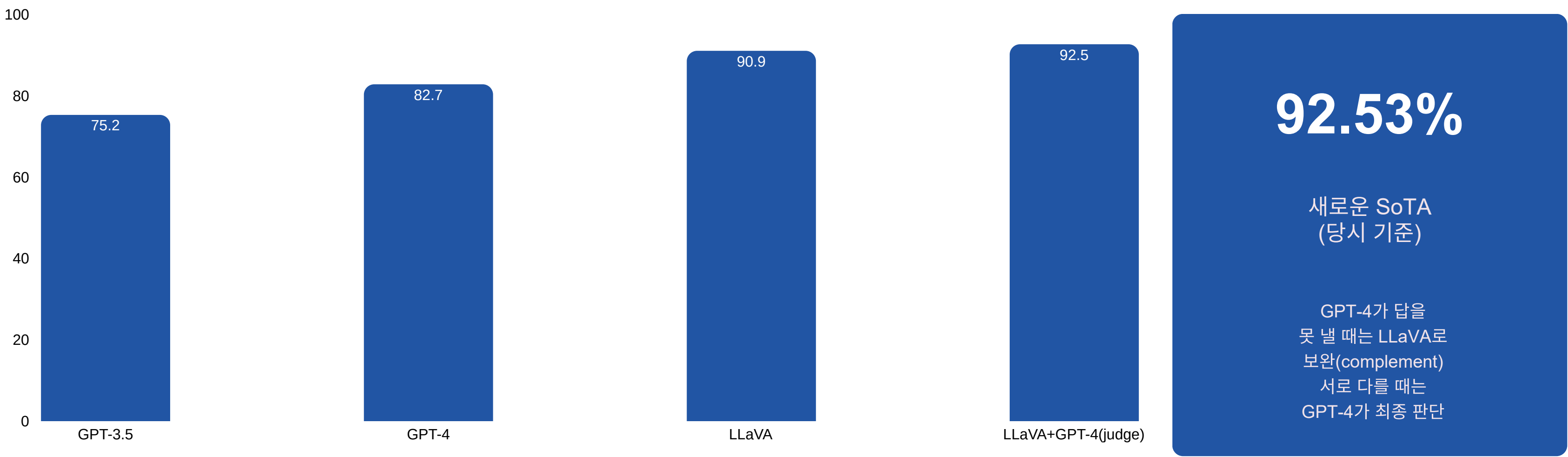


Experiments

- ScienceQA benchmark

❖ ScienceQA: 21K개 멀티모달 객관식 과학 문제

- GPT-4를 '심판'으로 결합(judge scheme)해 두 모델의 답이 다를 때 최종 판단



Ablation 핵심:

1. 마지막 이전 레이어 특징 사용이 더 좋음
2. 추론 → 정답 순서
3. Stage 1 생략 시 정확도 5.11%p 급락
4. 모델 클수록(7B→13B) 향상

Conclusion

❖ 결론

- Visual Instruction Tuning: GPT-4로 생성한 데이터로 이미지-언어 instruction-following 모델을 학습하는 접근
- 단순한 아키텍처(선형 projection)로도 사전학습된 컴포넌트를 잘 연결하면 강력한 성능이 나온다는 실증
- ScienceQA SoTA 달성(92.53%), LLaVA-Bench에서 GPT-4에 근접한 심층 이미지 이해 능력 입증

다음 논문(VGGT, SpatialStack)과의 연계점

- LLaVA는 CLIP의 2D 이미지 특징을 단순 선형 projection으로 언어모델에 연결 — 이미지를 "패치 토큰"로 다뤄 3D 공간 구조·기하학적 관계는 명시적으로 포착하지 못함
- 이 한계는 "2D 이미지를 넘어 실제 3D 공간을 이해하는 VLM을 만들려면 무엇이 필요한가?"라는 질문으로 이어짐
- 다음으로 살펴볼 VGGT(3D 기하 구조를 추정하는 geometry foundation model)와 SpatialStack(VGGT의 geometry feature를 VLM에 층별로 융합)은, 바로 이 "공간적 맥락 이해 부족"을 정면으로 다루는 후속 연구 흐름

Conclusion

- Comment

❖ LLaVA의 아쉬운 지점

단순 Linear Projection의 표현력 한계

Vision feature와 언어 임베딩을 1개의 선형 레이어로만 정렬
→ CLIP의 풍부한 시각 정보를 충분히 살리지 못할 가능성. (실제로 후속 LLaVA-1.5는 이 부분을 **2-layer MLP**로 교체해 성능을 끌어올림)

저해상도 Vision Encoder(CLIP ViT-L/14, 224×224)

작은 물체·텍스트처럼 세밀한 디테일을 포착하기 어려운 구조적 한계
→ 실제 논문 예시들도 장면 전반의 맥락 이해에는 강하지만 **세부 묘사**에서는 **한계**를 보임

GPT-4를 심판으로 쓰는 self-referential 평가

정량 평가(LLaVA-Bench) 자체가 GPT-4의 판단 기준에 의존
→ GPT-4가 선호하는 답변 스타일(장황하고 설명적인 톤)에 유리하게 **편향**됐을 가능성을 배제하기 어려움

158K instruction 데이터의 생성 파이프라인 의존성

데이터 품질이 전적으로 GPT-4의 캡션·bbox 해석 능력에 의존
→ GPT-4 자체의 편향(bias)이나 오류가 있다면 instruction data에도 그대로 전파될 위험

그럼에도 **"단순한 구조 + 좋은 데이터"**만으로 강력한 성능을 낼 수 있음을 보인 실증적 의의는 여전히 큼

PART II. VGGT

Visual Geometry Grounded Transformer

Motivation

- LLaVA의 한계에서 VGGT 필요성으로

❖ LLaVA는 왜 3D 공간을 이해하지 못하는가

- LLaVA는 이미지를 2D 패치 시퀀스로만 처리 — 깊이, 카메라 위치, 3D 구조 같은 공간 정보는 입력에 없음

❖ 3D 복원 방법론의 흐름



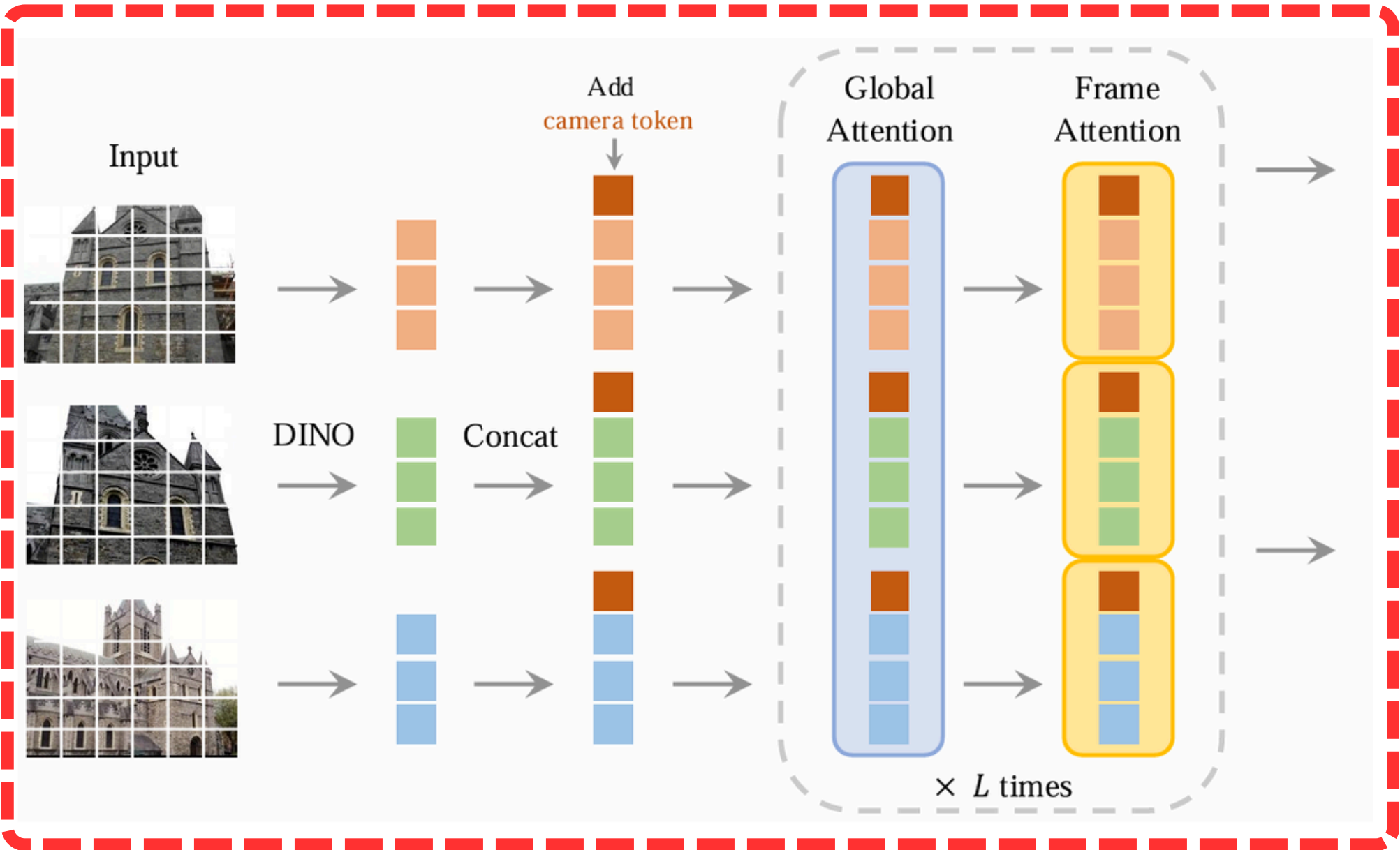
	기존 방식	VGGT
방식	Bundle Adjustment 등 반복 최적화 필요	단일 forward pass
처리	여러 단계(매칭→삼각측량→최적화)	한 번에 camera/depth/point map 동시 예측
속도(10장 기준)	DUS3R ~7초, MAS3R ~9초, VGGStM ~10초	약 0.2초

* Bundle Adjustment: 재투영 오차를 최소화하도록 카메라 포즈와 3D 점 좌표를 반복적으로 최적화하는 기법 · 속도는 IMC 벤치마크 기준 장면당 처리 시간

Architecture

❖ Alternating-Attention 구조

- Standard Self-Attention을 두 단계로 분리 하여 교차 수행
- 1. Frame - wise Attention : 이미지 내부 토큰 간 정보 교환 (Local)
- 2. Global Attention : 모든 프레임 전체 토큰 간 정보 교환 (Global)



DINOv2: 사전학습된 시각 표현을 재사용해 기하 추론에 학습을 집중 camera token: 카메라 포즈 예측용 학습 가능 토큰

➡ SpatialStack 모델이 사용하는 부분

	LLaVA	VGGT
이미지 처리 방식	각 이미지를 CLIP으로 독립 인코딩 → 단일 이미지 기준	여러 이미지를 함께 넣고, Frame/Global Attention을 번갈아 적용
목적	이미지 하나에 대한 언어 이해	여러 이미지 간의 3D 공간 관계 파악 (카메라 위치, 깊이 등)
이미지 간 정보 교환	없음 (이미지 하나씩 독립 처리)	있음 (Global Attention 역할)

"camera token도 patch token과 동일하게 L개 attention layer를 함께 통과
— 이 과정에서 이미지 내용을 흡수해 포즈 추론에 필요한 맥락을 축적

Ablation

- Ablation Study: Alternating-Attention

❖ 왜 Alternating-Attention 인가?

ETH3D Dataset	Acc.↓	Comp.↓	Overall↓
Cross-Attention	1.287	0.835	1.061
Global Self-Attention Only	<u>1.032</u>	<u>0.621</u>	<u>0.827</u>
Alternating-Attention	0.901	0.518	0.709

Table 5. Ablation Study for Transformer Backbone on ETH3D.

비교 조건

파라미터 수와 attention 레이어 수(2L)를 동일하게 맞추는 공정 비교

결과

Cross-attention은 프레임 간 융합을 최대화했는데도 가장 나쁨. 프레임 수가 늘수록 런타임도 급증

시사점

저자가 복잡한 cross-attention 대신 Alternating-Attention을 채택한 근거
— 단순한 구조가 오히려 더 정확함

핵심: 복잡한 cross-attention 없이, self-attention을 번갈아 쓰는 단순한 구조만으로 더 우수한 성능 확보

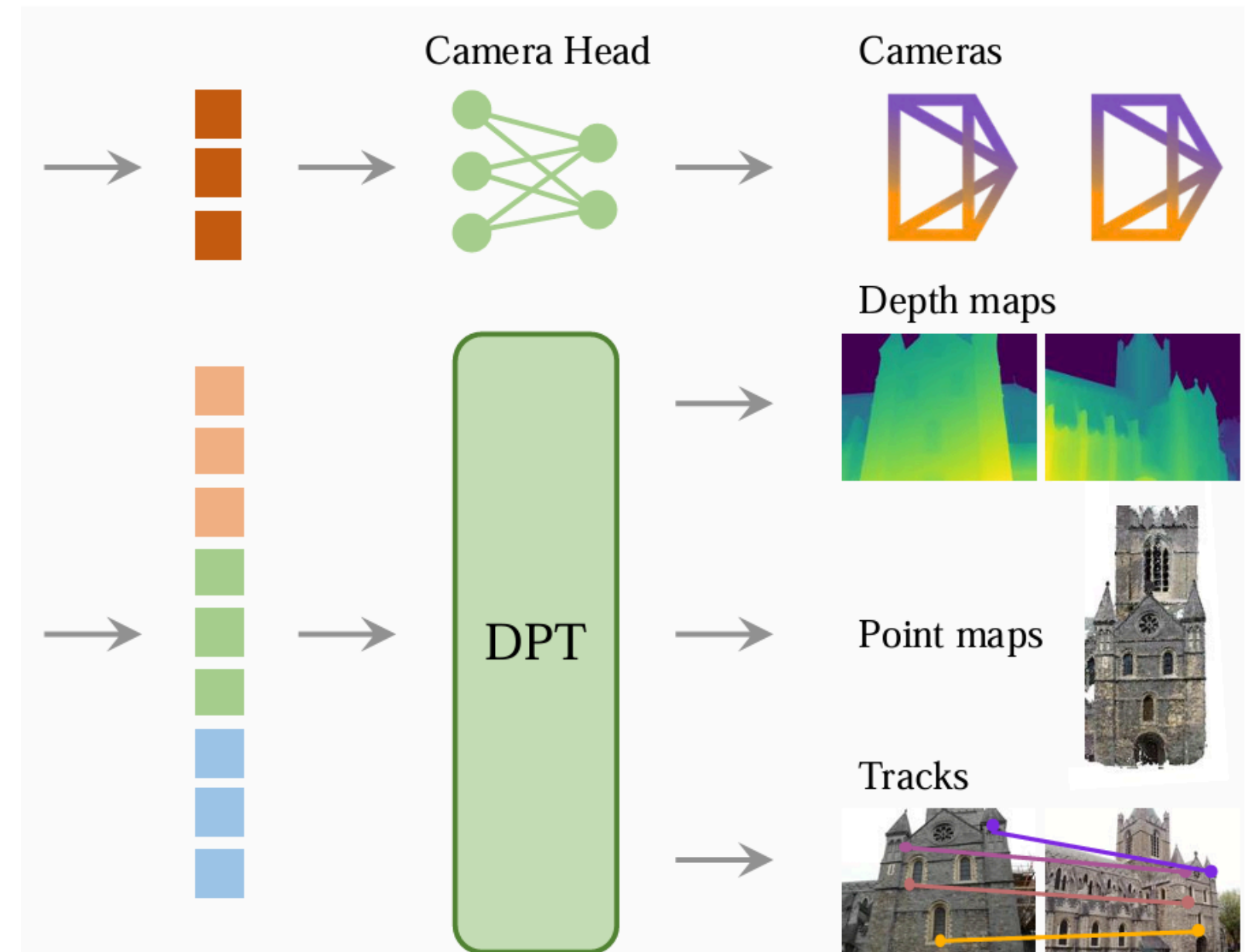
Architecture

- Prediction Head

❖ Camera Head 와 DPT Head

Alternating-Attention을 L번 통과한 표현은 두 갈래로 나뉘 (L층까지는 공유, 분리되는 이 시점부터)

1. Camera Head: camera token을 모아 self-attention + linear layer로 camera pose 예측
2. DPT: patch token을 dense feature map으로 변환해 depth map·point map·tracking feature 예측



두 head가 서로 다른 토큰을 입력으로 쓰기 때문에 한 번의 forward pass에서 병렬로 동작 → 4가지 3D 속성을 동시에 뽑아냄

Ablation

- Ablation Study: Multi-task Learning

❖ 왜 Multi-task Learning인가?

W. $\mathcal{L}_{\text{camera}}$	W. $\mathcal{L}_{\text{depth}}$	W. $\mathcal{L}_{\text{track}}$	Acc.↓	Comp.↓	Overall↓
X	✓	✓	1.042	0.627	0.834
✓	X	✓	<u>0.920</u>	<u>0.534</u>	<u>0.727</u>
✓	✓	X	0.976	0.603	0.790
✓	✓	✓	0.901	0.518	0.709

Table 6. Ablation Study on Multi-task Loss Composition.

비교 조건

point map 예측 기준, camera·depth·track loss를 하나씩 제외하며 ETH3D로 평가 (나머지는 VGGT와 동일)

결과

camera loss를 뺐을 때 성능 저하가 가장 큼(Overall 0.834) — 세 loss 모두 포함한 전체 학습이 0.709로 가장 우수

시사점

저자가 4개 head를 병렬 설계하고 함께 학습시킨 이유 — camera loss를 빼면 point map 정확도까지 하락, 멀티태스크 학습은 선택이 아닌 필수

핵심: 이 ablation이 VGGT가 4개 출력을 하나의 backbone에서 동시에 학습시키도록 설계된 이유를 직접 증명함

Architecture

- Bridge: VGGT → SpatialStack

❖ VGGT의 hidden state를 SpatialStack이 어떻게 재활용하는가



- VGGT는 3D 속성(camera·depth·point map·track)만 추정하고 복원하는 모델 → 언어모델과 결합하는 방법은 제시하지 않음
- SpatialStack은 VGGT를 재학습하지 않고, Camera Head와 DPT Head는 버린 채 내부 hidden state(L11 / L17 / L23)만 geometry feature로 재활용
- DPT 자체가 4 · 11 · 17 · 23번째 층에서 정보를 뽑도록 설계됨 — 층마다 담긴 정보가 다르다는 것이 PART III(SpatialStack)의 출발점

그럼 이 3D 표현을 언어모델의 어디에, 어떻게 넣을 것인가? → PART III. SpatialStack

PART III. SpatialStack

Layered Geometry-Language Fusion for 3D VLM Spatial Reasoning

paper review

목차

- Introduction
- Why Multi-level Geometry Features?
- Architecture: Geometry-Language Fusion
- Experiments
- Conclusion

SpatialStack: Layered Geometry-Language Fusion for 3D VLM Spatial Reasoning

Jian Zhang^{1*} Shijie Zhou^{2,3*} Bangya Liu^{4*} Achuta Kadambi² Zhiwen Fan⁵

¹XMU ²UCLA ³Google ⁴UW-Madison ⁵TAMU

<https://spatial-stack.github.io/>

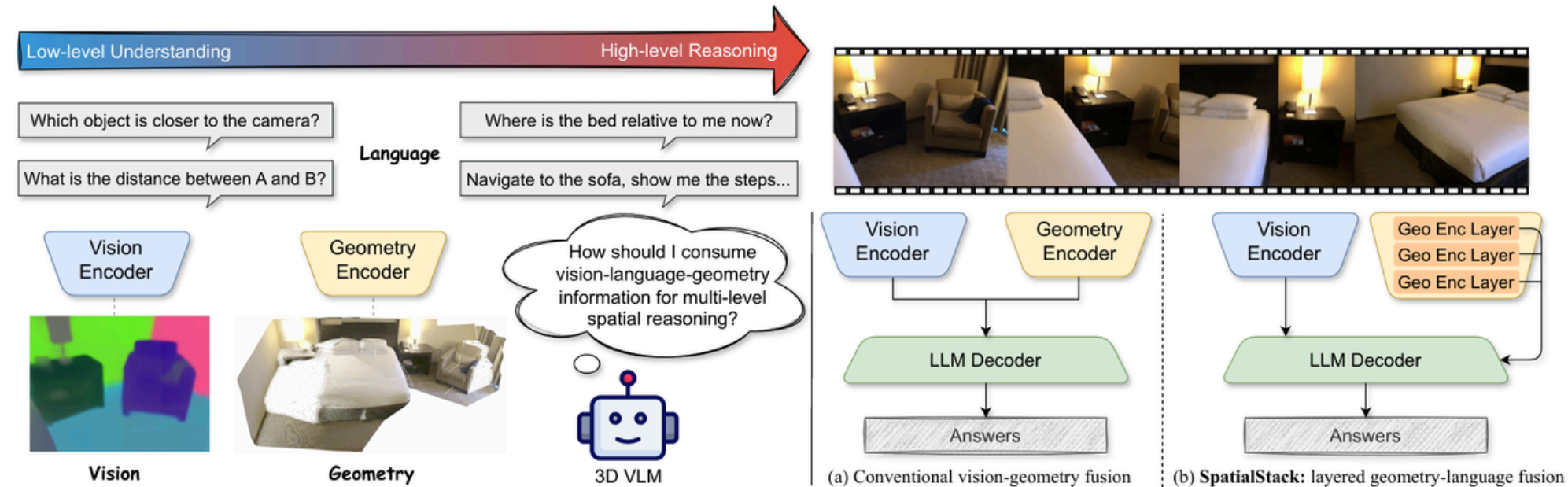


Figure 1. **SpatialStack: Layered Geometry-Language Fusion.** Conventional VLMs (a) fuse only a single deep geometry feature with vision tokens, which limits both fine-grained spatial understanding and high-level spatial reasoning. SpatialStack (b) instead stacks multi-level geometry features and injects them hierarchically into successive LLM decoder layers, yielding stronger 3D spatial understanding across benchmarks.

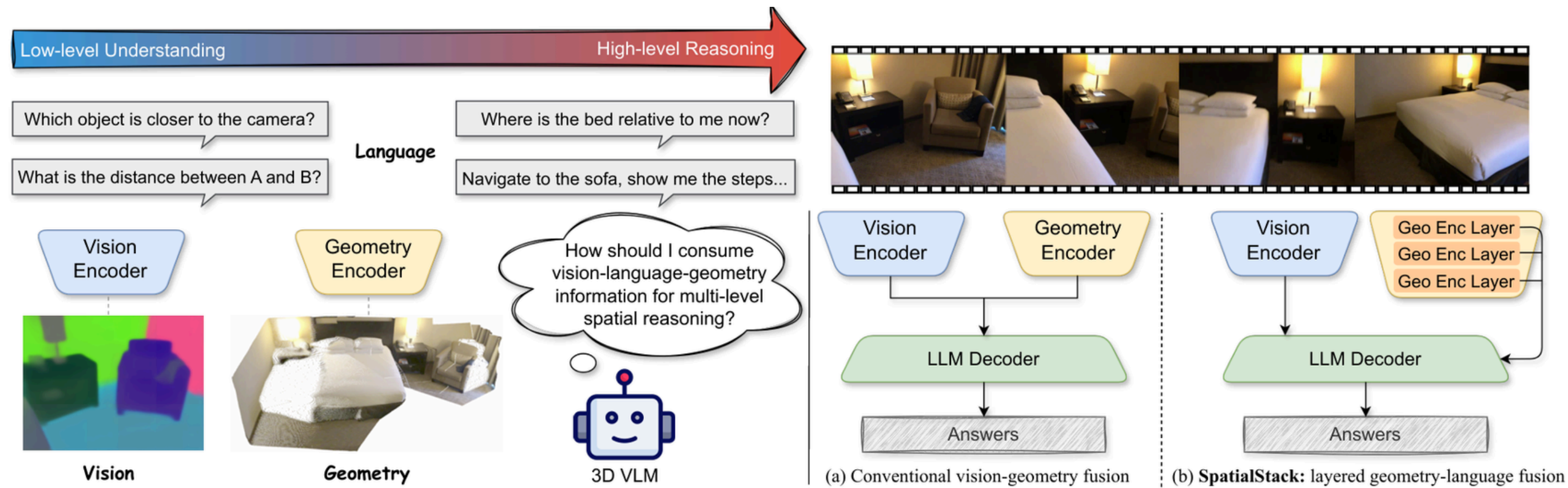
- 2026년 CVPR에 게재 (XMU, UCLA, Google, UW-Madison, TAMU 공동 연구)
- VGGT를 geometry encoder로 활용해 VLM의 3D 공간 추론 능력을 강화하는 계층적 융합 프레임워크
- 기존 연구들이 “가장 깊은 층 하나만” 융합하던 한계를 지적하고, 여러 층을 LLM decoder에 계층적으로 주입
- LLaVA(vision+LLM 결합)의 구조를 계승하면서, VGGT라는 3D geometry encoder를 추가로 결합한 확장판

Introduction

- Motivation

❖ 기존 VLM들이 3D 공간 추론에 약한 이유

- 대형 VLM들은 3D geometry·공간 관계를 제대로 인코딩하지 못함
- 최근 연구들이 geometry transformer(VGGT 등)를 VLM에 결합하기 시작했지만, 공통적으로 가장 깊은 층(deep layer)의 feature 하나만 융합
- 그러나 geometry encoder는 DPT 구조 기반 — 원래 여러 층에서 정보를 뽑아 쓰도록 설계됨, 마지막 층만 쓰면 중간층의 풍부한 계층적 신호를 버리게 됨



기존 방식
deep layer feature만 융합



한계
세밀한 지역 정보 손실



결과
공간 추론 병목 현상 발생

Introduction

- Contributions of this paper

❖ 이 논문의 3가지 핵심 기여

①

Layer-wise 분석 최초 시도

vision·geometry encoder와 LLM decoder 간 융합 깊이가 공간추론 세분화에 미치는 영향을 체계적으로 규명

②

SpatialStack 프레임워크

여러 층의 geometry feature를 LLM decoder에 계층적으로 정렬·주입 — 기존 단일 심층 융합을 넘어서는 설계

③

VLM-SpatialStack 구현

Qwen 계열 기반 구체화, 여러 3D 벤치마크에서 SOTA 및 강한 일반화 성능 입증

Why Multi-level Geometry Features?

- Qualitative Analysis

❖ 왜 deep layer feature 하나만으로는 부족한가

- 한 시점의 이미지를 VGGT에 넣고, 서로 다른 층의 patch token을 원래 H×W 격자 형태로 다시 펼쳐 시각화
- 특정 영역(ROI)과 다른 모든 위치 간의 유사도를 계산 — 밝을수록 유사

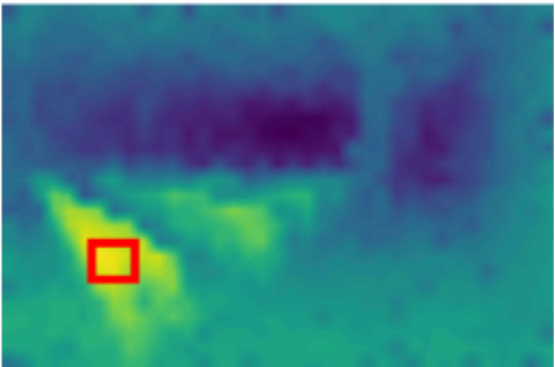
Shallow Layer

- 경계가 뚜렷하고 선명함, 지역 구조가 잘 보존됨
→ 세밀한 저수준(low-level) 공간 지각에 유리

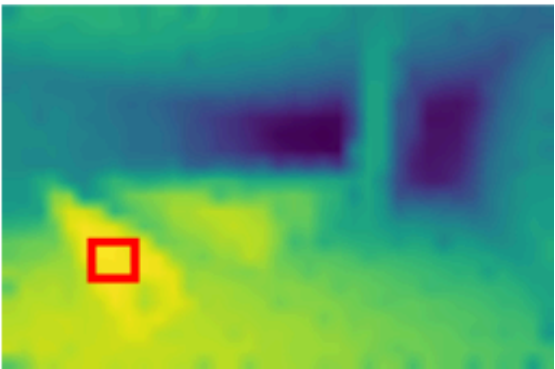
Deep Layer

- 활성화가 균일해져 물리적으로 다른 영역도 비슷해 보임(세밀 구조 손실)
→ 대신 전역적/의미적 맥락을 압축 → 고수준(high-level) 추론에 유리

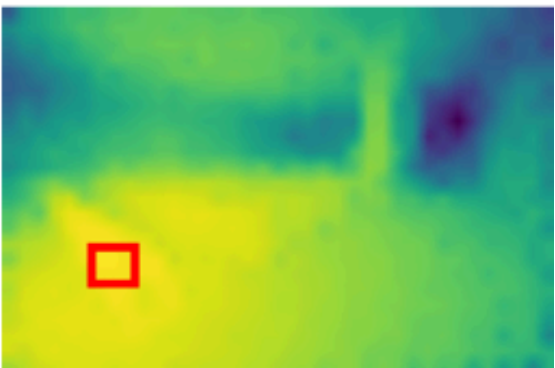
Shallow Layer



Middle Layer



Deep Layer

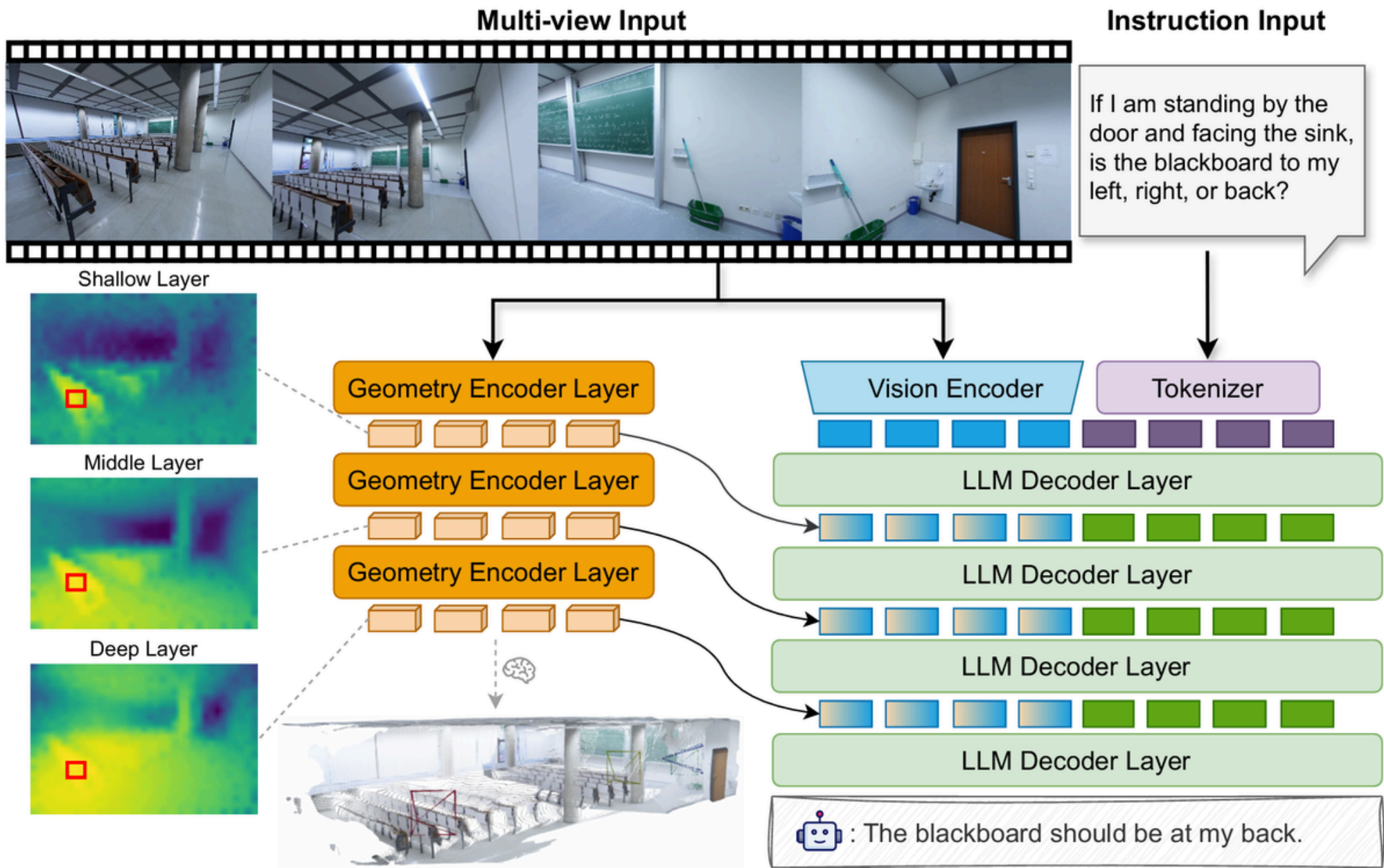


shallow·deep feature가 서로 다른 역할 → '여러 층을 동시에 활용하자'는 SpatialStack의 출발점

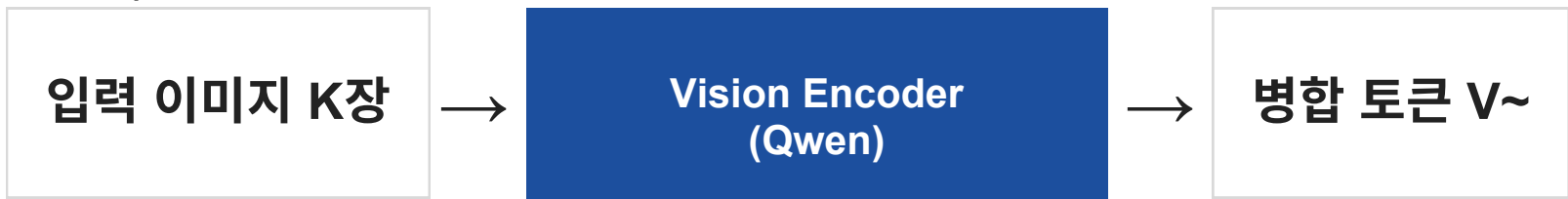
Architecture

- Full pipeline

❖ SpatialStack 전체 구조



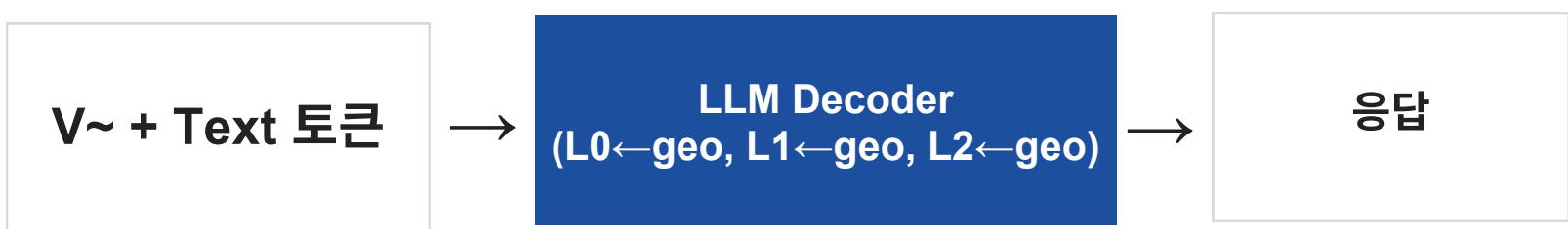
Vision path



Geometry path



↓↓↓ (각 층별 additive residual로 주입)



Architecture

- Why Multi-level Geometry Features - Quantitative Analysis

❖ 레이어별 주입 실험 + 그래프로 본 경향

Model	Low-Level Avg	High-Level Avg	Overall
Qwen3.5 (fine-tuned)	61.37	64.76	63.07
Single-layer (geo enc: 11)	66.11	64.48	65.30
Single-layer (geo enc: 17)	63.89	65.76	64.83
Single-layer (geo enc: 23)	64.33	66.36	65.35
Multi-Layer Fusion	64.69	65.15	64.92

Table 1. Ablation Results on Geometry Token Fusion Depth.

- 의외로, 여러 층을 그냥 합치는 Multi-Layer Fusion(64.92)이 가장 좋은 단일 층 L23(65.35)보다도 낮음
- 이 관찰이 바로 SpatialStack 설계의 직접적 동기

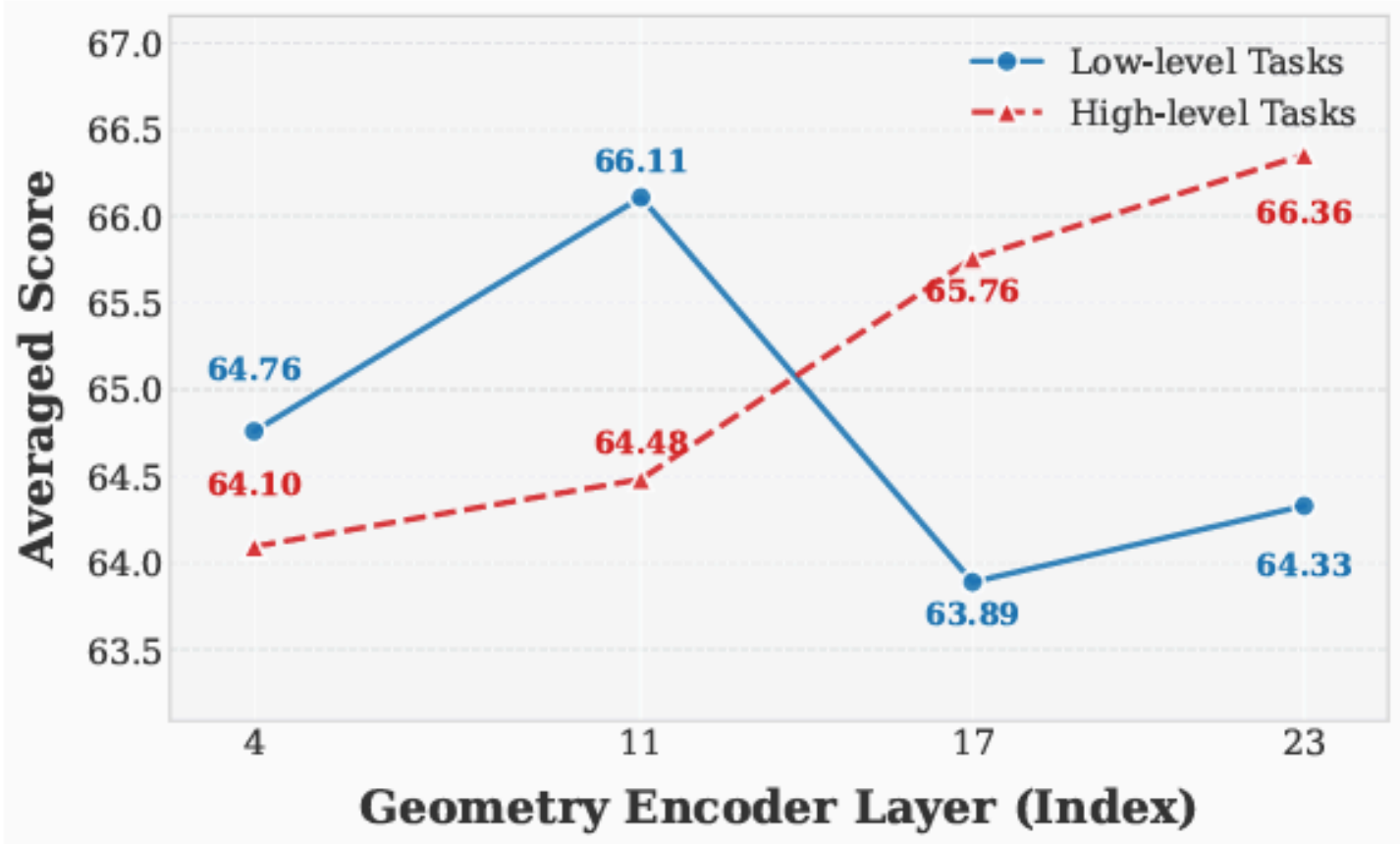


Figure 4. Effect of Geometry Injection Layers on Spatial Tasks.

왜 단순 합산은 실패할까?

- 얇은 층(국소적 구조)과 깊은 층(전역적 의미)은 서로 다른 종류의 정보를 담고 있어, 그대로 더하면 두 신호가 서로 간섭·상쇄될 수 있음
- Figure 4에서 보듯 low-level과 high-level의 최적 깊이 자체가 다름 → 하나의 결합 방식으로 평균을 내면 어느 쪽에도 최적화되지 않음

Low-level은 L11에서 정점 후 하락, High-level은 깊을수록 꾸준히 상승 — 서로 다른 최적 깊이를 가짐

Architecture

- Geometry Encoder

❖ Geometry Encoder = VGGT (Frozen)

VGGT — 24개 Alternating-Attention 레이어 (frozen)

L11

L17

L23

Eq. (3) — VGGT 내부 L개 레이어를 통과한 최종 표현

$$\mathbf{Z}_L = f_L^{\text{geo}} \left(f_{L-1}^{\text{geo}} \left(\cdots f_1^{\text{geo}} \left([\mathbf{Z}_0^{(1)}; \cdots; \mathbf{Z}_0^{(K)}] \right) \right) \right)$$

- VGGT를 3D 속성 예측 모델이 아닌 “특징 추출기”로 재해석 — DPT head 대신 중간 hidden state만 추출
- VGGT 내부 24개 레이어 중, 세 개의 지점(L11 / L17 / L23)을 선택해 geometry feature로 활용
- VGGT 자체 DPT가 쓰던 4개 지점(4/11/17/23) 중, 가장 얇은 4번만 제외 (예비 실험에서 성능이 낮았기 때문)
- 왜 DPT head 대신 hidden state인가?
DPT head는 depth·point map 등 특정 예측 task에 맞춰 압축된 출력 — 범용성이 떨어짐
* hidden state는 task-specific하게 좁혀지기 전 단계라 VLM에 재사용하기 더 유연함

Architecture

- Geometry-Language Fusion

❖ 계층적 주입: Geometry layer ↔ LLM decoder layer 대응

Eq. (4) — Geometry token merger

$$\mathbf{G}_{l_i} = \mathcal{M}_{\text{geo}}^{(l_i)}(\mathbf{Z}_{l_i}), \quad \mathbf{G}_{l_i} \in \mathbb{R}^{N' \times D_{\text{lang}}}.$$

Eq. (5) — LLM decoder layer j에 additive residual로 주입

$$\mathbf{H}^{(j)'} = \mathbf{H}^{(j)} + \mathbf{G}_{l_j}, \quad j \in \{0, 1, 2\}.$$

Geometry Layer	LLM Decoder Layer	대응 의미
L11 (shallow)	L0	세밀한 저수준 정보 → 언어모델 초반에 주입
L17 (middle)	L1	중간 수준 맥락 → 언어모델 중반에 주입
L23 (deep)	L2	전역적 의미 맥락 → 언어모델 후반에 주입

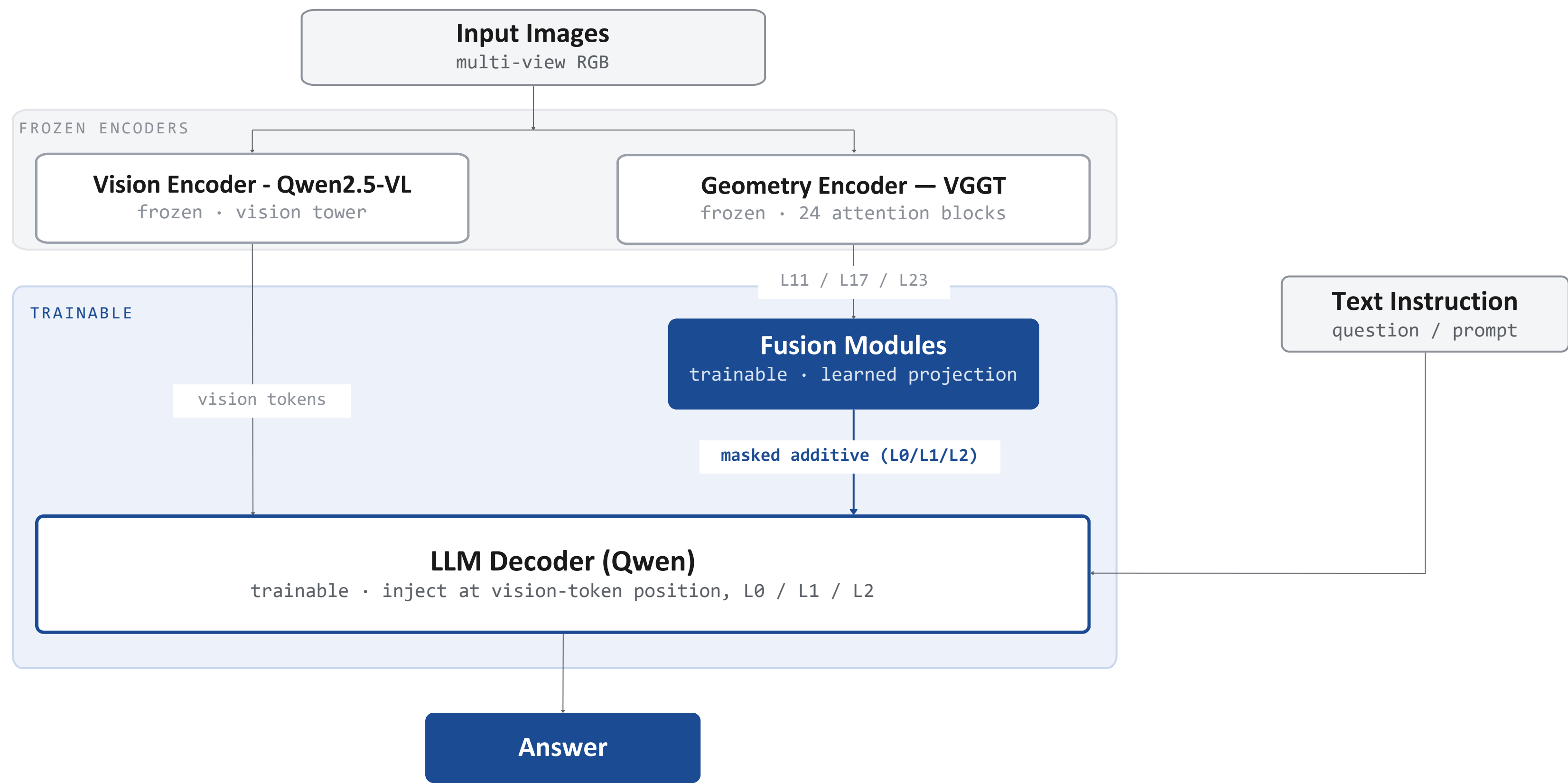
왜 concat이나 cross-attention이 아니라 masked additive fusion인가?

- LLM decoder 구조 자체는 그대로 유지하면서 값만 더하는 방식이라 추가 파라미터·연산 오버헤드가 거의 없음
- vision-token이 위치한 자리에만 정확히 더해지므로, text-token 경로 (언어 능력)는 건드리지 않고 geometry 정보만 선택적으로 주입 가능

Masked additive fusion: geometry feature는 vision-token이 위치한 자리에만 더해짐 (text token에는 영향 없음)

Architecture

- Full Pipeline Overview: Input → Geometry-Language Fusion → Answer



Architecture

- Implementation Details (Appendix A)

❖ Geometry token을 vision token과 어떻게 정렬하는가

1. Token Structuring: VGGT 각 레이어 출력은 camera token 1개 + register token 4개 + patch token들로 구성 — fusion 전 camera/register token은 제거하고 patch token만 사용
2. Windowing/Reordering: patch token을 vision encoder의 병합 토큰과 spatial하게 맞추기 위해, $s \times s$ (기본 $s=2$) 윈도우 단위로 재정렬 — 인접한 4개 geometry token이 병합된 1개 vision token과 대응
3. Geometry-to-Language Projection: RMSNorm으로 정규화 → 윈도우 단위 병합 → 2-layer MLP로 언어모델 차원에 맞춤

핵심: geometry token과 vision token의 "공간적 위치"를 정확히 맞춰야 masked additive fusion이 올바른 위치에 정보를 더할 수 있음

Experiments

- VSI-Bench Detailed Breakdown

❖ VSI-Bench 세부 태스크별 성능 (Table 3)

- Rank는 그룹별 산정이지만, 원점수 기준으로는 Gemini-2.5 Pro(51.5)까지 포함해 SpatialStack-5B(67.5)가 최고점
- Route Plan에서 격차 압도적(오픈소스 2위 대비 +22.2p) — 이 태스크는 학습 데이터에도 없었는데 달성한 결과라 제로샷 일반화 성능의 증거

Methods	Rank	Avg.	Obj. Count	Abs. Dist.	Obj. Size	Room Size	Rel. Dist.	Rel. Dir.	Route Plan	Appr. Order
			Numerical Answer				Multiple-Choice Answer			
Baseline										
Chance Level (Random)	-	-	-	-	-	-	25.0	36.1	28.3	25.0
Chance Level (Frequency)	-	34.0	62.1	32.0	29.9	33.1	25.1	47.9	28.4	25.2
Proprietary Models (API)										
GPT-4o	2	34.0	46.2	5.3	43.8	38.2	37.0	41.3	31.5	28.5
Gemini-2.5 Pro	1	51.5	43.8	34.9	64.3	42.8	61.1	47.8	45.9	71.3
Open-source Models										
LongVILA-8B	15	21.6	29.1	9.1	16.7	0.0	29.6	30.7	32.5	25.5
Qwen2.5-VL-3B	14	28.7	33.1	19.4	17.4	24.8	37.3	44.3	31.4	22.0
VILA-1.5-8B	13	28.9	17.4	21.8	50.3	18.8	32.1	34.8	31.0	24.8
LongVA-7B	12	29.2	38.0	16.6	38.9	22.2	33.1	43.3	25.4	15.7
VILA-1.5-40B	11	31.2	22.4	24.8	48.7	22.7	40.5	25.7	31.5	32.9
LLaVA-OneVision-7B	10	32.4	47.7	20.2	47.4	12.3	42.5	35.2	29.4	24.4
LLaVA-Video-7B	9	35.6	48.5	14.0	47.8	24.2	43.5	42.4	34.0	30.6
LLaVA-OneVision-72B	8	40.2	43.5	23.9	57.6	37.5	42.5	39.9	32.5	44.6
LLaVA-Video-72B	7	40.9	48.9	22.8	57.4	35.3	42.4	36.7	35.0	48.6
Spatial-MLLM-4B	6	47.0	65.3	34.8	63.1	45.1	41.3	46.2	33.5	46.3
VG-LLM-4B	5	47.3	66.0	37.8	55.2	59.2	44.6	45.6	33.5	36.4
Qwen3.5-4B	4	53.6	56.5	36.5	67.5	53.8	60.3	57.5	34.0	62.3
Cambrian-S-3B	3	57.3	70.7	40.6	68.0	46.3	64.8	61.9	27.3	78.8
SpatialStack-4B (Qwen2.5)	2	60.9	69.2	45.4	63.0	63.2	57.9	68.4	40.2	79.6
SpatialStack-5B (Qwen3.5)	1	67.5	71.0	55.6	69.1	68.2	67.3	84.1	41.2	83.5

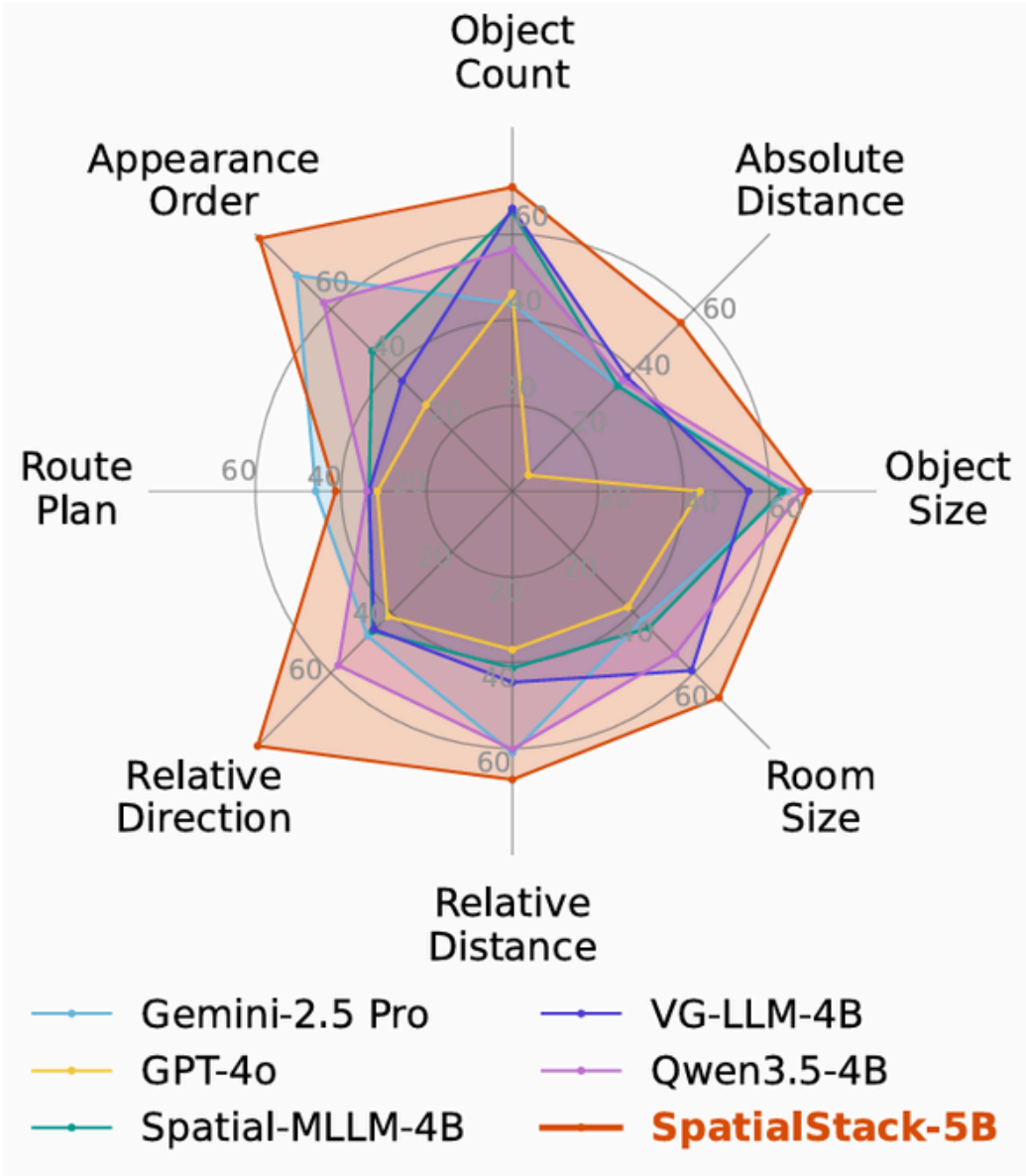


Table 3. Evaluation on VSI-Bench.

Ablation

- Ablation: Fusion Order (Table 7)

❖ 계층적 정렬 순서가 정말 중요한가?

Methods	VSI Bench	SPAR Bench	BLINK Spatial	CV Bench	Overall
Qwen3.5	64.76	68.75	56.10	84.49	68.52
Vision Fusion	64.27	69.68	56.45	83.11	68.38
SpatialStack (Reverse)	67.22	71.97	50.08	84.82	68.52
SpatialStack (final)	67.52	71.39	52.12	85.53	69.14

Table7. Geometry-LanguageFusionOrderAblation.

- SpatialStack(final): Geo-L11→LLM-L0, Geo-L17→LLM-L1, Geo-L23→LLM-L2 (shallow→shallow, deep→deep)
- SpatialStack(Reverse): 순서를 뒤집음(Geo-L11→LLM-L2, ..., Geo-L23→LLM-L0)
- 실무적 의미: 구현 시 대응 순서를 임의로 바꾸면 성능 손실 — shallow↔shallow, deep↔deep 매칭을 반드시 지켜야 함

정방향(final)이 Reverse·Vision Fusion보다 우수 → 얇은 층은 얇은 층에, 깊은 층은 깊은 층에 대응시키는 계층적 정렬 자체가 핵심

Conclusion

❖ 결론

- shallow layer는 세밀한 저수준 지각, deep layer는 전역적 맥락 → 계층적 대응관계를 규명
- 단순한 다층 융합은 feature interference를 유발 → 융합 위치와 방식(순서)이 핵심 과제
- VGGT의 다층 표현을 LLM decoder에 계층적으로 주입 → 여러 3D 공간추론 벤치마크에서 SOTA, 일반 능력도 유지

VGGT와의 연계점

- VGGT는 3D 속성(camera/depth/point map/tracking)을 직접 예측하는 24층 feed-forward transformer
- SpatialStack은 VGGT의 예측 head(DPT)는 버리고, 내부 24층 중 3개 지점(L11/17/23)의 hidden state만
“**geometry feature 추출기**”로 재활용
- VGGT가 검증한 “멀티태스크 학습이 3D 이해를 돕는다”는 통찰
→ SpatialStack은 “멀티레이어가 spatial reasoning의 서로 다른 수준을 돕는다”는 통찰로 확장

Conclusion

- Final Comment

❖ LLaVA → VGGT → Spatial Stack 최종 코멘트

"VLM이 세상을 어떻게 이해해야 하는가?"에 대해, 세 논문은 한 단계씩 답을 확장

1. **LLaVA**: "이미지를 언어모델이 이해하게 하려면?" → CLIP+선형 projection

*이미지를 "패치 뭉치"로 취급 — 물체가 얼마나 가까운지, 카메라가 어디서 찍었는지는 알 수 없음

2. **VGGT**: "그럼 3D 기하 정보는 어떻게 얻는가?" → 반복 최적화없이 feed-forward 한 번으로 camera/depth/point map을 뽑아냄.

*3D 추정 모델일 뿐, 언어모델과 결합하는 방법은 제시하지 않음

3. **SpatialStack**: "그럼 그 3D 정보를 언어모델에 어떻게 넣어야 하는가?" → VGGT의 주요 3가지 레이어를 언어모델 여러 층에 계층적으로 주입. 단순히 합치면 오히려 성능이 떨어진다는 걸 실험으로 증명하며, "어디에 어떻게 넣는가"가 핵심 과제를 보임

남아있는 한계

1. **Vision encoder는 그대로** : CLIP, Qwen encoder 모두 세 논문 내내 재설계되지 않고 옆에 뭔가 추가하는 식으로만 우회함

2. **정보를 거르지 않고 전부 주입** : 질문이 뭐든 상관없이 vision/geometry 정보를 똑같이 밀어넣음 → 질문에 따라 필요한 정보만 선별하는 메커니즘 필요

Q&A

상명대학교 소프트웨어학과 이준형

leejunhyeong02@gmail.com

010-3205-3276